

Tab 1

The Lindy PM Standard of Performance

Flo | 2024.01.01

This is the “standard of performance” we wrote for PMs at Lindy, lightly edited for external consumption. If you’re here for very tactical advice, jump straight to “[Holding the Bar](#).”

About This Document

This is going to be a long doc!

It’s inspired by Bill Wash’s excellent “[The Score Takes Care of Itself](#)”, which exhorts you to *focus on your inputs*. If you do that, and get every single one of these, even the most minute, exactly right, then the output (“the score”) will take care of itself.

It follows that you should be excruciatingly clear with everyone on what *exactly* is expected of them. It’s hard enough to do a good job when you know exactly what your job is; it’s impossible if that was unclear to begin with.

So, this is what this document is about — I’ll lay out:

1. Your high-level mission — the output we’re going after
2. The activities you’ll perform in pursuit of this mission — the input
3. What your first few weeks will look like

Mission

Ship a « holy shit » product that, long-term, brings us closer to our vision of the AI employee and, short term, builds a growing business.

Responsibilities / Activities

You must ensure that we (in this order):

1. Build the right thing
2. Build it right
3. Build it fast

”Build the Right Thing”

This is first on the list for a reason — nailing execution doesn’t matter if you’re working on the wrong thing.

To “build the right thing,” you must first build a solid point of view around:

1. **The category** we are playing in (no-code workflow automation)
2. **The opportunity** that already existed within it before AI
3. **How it is impacted** by recent changes in AI
 - a. Which new use cases can be served?
 - b. How can existing ones be served better?
4. **The state of the union** at Lindy
 - a. What’s working? What isn’t?
5. **The most important things** to work on next
 - a. What must urgently be fixed in order to bridge our “performance gap?”
 - b. What must importantly be built in order to bridge our “opportunity gap?”

These two terms were coined by HBS professor Linda Hill.

In a nutshell, bridging your performance gap means taking care of your existing business; if you’re Apple in 2000, your performance gap is the Mac. How do you keep taking market share in the PC industry?

Bridging your opportunity gap means pursuing the things your business *could become*. For Apple in 2000, this was the iPod.

You never run out of performance gap to bridge, but you must still find a way to bridge the opportunity gap — because if you don’t do it, someone else will, and it will become your performance gap.

You will build this point of view through:

1. **Most importantly, direct exposure:** spend a ton of time in the product to automate workflows of varying complexity.
 - a. Start small and gradually build more and more powerful Lindies — it is important that you stretch the product and get a first-hand feel for its limits.
 - b. As you use the product, please do keep a detailed « friction log » — do read [this excellent explainer](#) if you’re unfamiliar with friction logs. Then, send it to the team at the end of your first week.

2. **Conversations with X, Y, and me** — the three people in the company who are closest to the customer.
3. **Meeting with customers**, in particular our top ones.
 - a. Mission #1 on day 1: get the list of top 20 customers from X.
 - i. Build a Lindy to send an email to all of them, asking to meet; sending 3 reminders over a week if no reply, and if they do reply and say yes, cc'ing your scheduling Lindy to find times.
 - b. As you meet with these customers, do tag me along (I love meeting users) and invite anyone else to join if they'd like.
 - c. What you're trying to learn:
 - i. Who are they?
 - ii. What problem are they trying to solve?
 - iii. Why us? What else have they tried? How did it fall short? How did they hear about us?
 - iv. What do they like so far? What do they dislike?
4. **Meeting with prospective customers.**
 - a. Ask X to add you to her implementation calls.
 - b. X is probably our top power user, and also a great salesperson.
 - c. You'll learn a ton in these calls — take lots of notes, and be prepared to share them at the next all-hands.
5. **Answering support tickets** and messages in [our Slack community](#).
 - a. Generally recommend you spend a ton of time in the community.
 - b. My dream is that eventually everyone knows you by name there!
 - c. Ask X to be added to the customer support rotation.

Throughout this whole exercise, please do:

- Ask lots of dumb questions.
- Talk about the ugly stuff — it's hard enough to solve problems when we talk about them, it's impossible if we don't.
- Take a lot of notes and send them around.

“Build it Right”

“Build it right” comes before “build it fast” for two reasons.

First, I just don't want to build a company that ships half-baked products to see what sticks.

Second, customers care more about quality than speed: if your product is crap, nobody cares how fast you shipped the crap. You can [read more about my views on this here](#).

Building the product right comes through a combination of 4 critical ingredients:

1. **Taste.** This is the most important component, and I think it is tragically underrated by modern PMs. People sometimes think they can build a great product just by being “data-driven” — as if a spreadsheet could ever be a substitute for a brain.

For an indication of where that can lead, consider that the most data-driven companies — Google and Amazon — also build some of the worst products. And that the company that builds the best products in the world (I don’t need to name them) barely even has access to usage data internally.

I think honing this taste and product sense is a product person’s most important responsibility. And as David Hume argued in “[The Standard of Taste](#),” taste is a skill like any other, that can be refined.

I would volunteer a few suggestions on how to do that:

1. **Listen to your inner voice.** It sounds cheesy, but it’s true: so many people simply don’t pay attention to whether they even *like* what they’re shipping.

This goes such a long way towards building a great product: does it make *your* heart sing?

2. **Pay attention to the details.** “God is in the details,” as Steve Jobs said. That’s all your product is: a collection of details. And even if the customer doesn’t see each of these individual details, they can feel their collective presence.
3. **Notice (and use!) other great products.** Pay attention to details not just in Lindy, but in every product you use day to day.

You’ll see us post details of products we like in the #design channel on Slack every so often — you’re highly encouraged to do the same!

4. **Maximize contrast.** One of the easiest ways I know to make product decisions is to think in extremes.

For example, when considering whether to introduce a setting: which feels best between a product with *tons of settings* everywhere, and one with *no settings*? I think the latter is much better. That of course doesn’t mean that we should have no settings — just that we should generally be reluctant to do so.

5. **Dare be opinionated.** The 5 most dangerous words in product are “let’s run an A/B test.”

A/B tests are perfect for optimization problems, like when you want to maximize a

funnel's throughput.

But when you're making a qualitative decision, the best tool you have is your taste and intuition, honed over thousands of hours building expertise in your user and domain (see below).

2. Exposure to users. Jared Spool wrote this amazing article I refer to all the time: [Fast Path to a Great UX](#). Quoting from it: (emphasis mine)

"As we've been researching what design teams need to do to create great user experiences, we've stumbled across an interesting finding. **It's the closest thing we've found to a silver bullet when it comes to reliably improving the designs teams produce.** This solution is so simple that we didn't believe it at first. After all, if it was this easy, why isn't everyone already doing it?

[...]

The solution? Exposure hours. **The number of hours each team member is exposed directly to real users interacting with the team's designs or the team's competitor's designs.** There is a direct correlation between this exposure and the improvements we see in the designs that team produces."

The implication is obvious: spend a ton of time not just talking with users, but watching them use the product!

(Every time I do that, I think of this meme — "[how it feels to watch a user use your product.](#)")

It's important to emphasize the deeper reason *why* exposure to users leads to better products: because it helps you build a better mental model of the user, which ultimately means you'll be able to take a lot more decisions every day *without* needing to speak to

them. Patrick Collison put it perfectly:



- 3. Deep understanding of the field.** That is the “strong mental model of the domain” mentioned above. Make sure you understand the technology cold.

In general, when you’re in a meeting with engineers and they use a technical term you don’t understand, either ask them about it, or make a note to ask your favorite LLM.

- 4. Iteration.** Tight iteration loops are so powerful that they can be adequate substitutes for a brain. The most spectacular example of this is natural selection — a completely brain-less process, operating off pure iteration, which gave rise to the most complex systems in the world.

It is impossible to build a great product from the top of your ivory tower: a product is a complex system, so the only way to design a great one is to try something, see how it falls short, make a change — rinse and repeat.

On this topic, I highly recommend you watch [Tom Chi's "Rapid Prototyping" talk](#). My favorite idea from it is “stay close to the medium:”

“A painter's medium is oil paint, and they stay constantly in contact with it — mixing colors, applying strokes, scraping with palette knives. Their hands are always in the medium. That's how great work gets done.

But imagine if you were a painter, but every time you wanted to make a brushstroke, instead of simply dabbing paint and applying it to canvas, you had to: stop, write a slide deck, take it to a meeting, communicate to get shareholders on board, formalize it in a specification, take that to dev managers who argue it down and decide to build 40% of it, then QA teams say it's not acceptable, and after a couple months of this process, only then you get to make your brushstroke.

Think about how hard it would be to create a masterpiece if that's how you had to paint.”

This “tight iteration loop” principle is why you’ll often hear us ask what we can “[ship by Friday](#).” Get the product to a point where you can start playing with it — even if it’s rough — within literally a week. This has the added advantage of building momentum and improving team morale.

5. **High expectations.** It is truly surprising how 90% of doing great work is simply *deciding to*. The phenomenon is so consistently documented that it has a name: [the Pygmalion Effect](#) — high expectations lead to high performance, and low expectations to low performance.



Flo Crivello ✓ @Altimor · Jan 19, 2023

...

It's really striking how much of the world and most objects around us suck — not because people are **unable** to do good work, but because they're **unwilling** to

Simply **deciding** to do outstanding work is 90% of the way there

8

22

230

39K

...



Flo Crivello ✓

@Altimor

...

Hold everything constant — people, capital, everything

Group A, you inject a Steve Jobs in — a maniac who keeps throwing your work back at you until it's perfect

And group B, you just let them do their thing

I guarantee group A will produce 10x better output, w the same input

Such a good tweet, I wonder who posted it

This makes the job of a PM almost easy: all you have to do is hold the bar!

Holding the Bar

This is such an important idea that it's worth dedicating a section to lay out *exactly* what “holding the bar” entails, at different stages of the product lifecycle:

1. **In product jams**, while the bones of the product are being thought through, you must ask yourself: are we thinking big enough? Have we considered all options? Do we understand the job to be done deeply enough? I love this quote from Frank Slootman:

“[You don't want okay.] Okay is terrible. You want great, you want fantastic, you want wow.”

I often find that what most prevents people from getting to this “wow” is self-limitation.

This is why our “refuse to accept constraints” value (inspired by Paypal) is so important: all magical experiences start with the decision to do something that everyone else deemed impossible.

The product jam is the very first part of our product lifecycle, and we start it with the most important part of the whole project: when we align on its job to be done.

Steve Jobs said that “design is the *fundamental soul* of a man-made creation that ends up expressing itself in successive outer layers of the product or service” (emphasis mine). This is why it’s so important to spend time getting this right from the get go: the job to be done is the soul of the product, and everything else is downstream from it.

2. **In design reviews**, while we concretely decide exactly what we will build. Do you feel that we’ve “[crafted the details](#)?”

Animate those icons. Brand that scrollbar. Polish that `:active` state. Load a single typeface glyph to render the world’s best ampersand. Make it fast. Make it accessible. Add keyboard shortcuts. Add tooltips. Animate the transition between two open tooltips. Manually measure text and avoid line wrapping widows. Design favicons that reflect app state. Convert internal URLs to rich previews. Render a custom `<input />` text caret and animate it better than any web browser does. Design a whole new stylesheet for printing. Design a whole new page for mobile. Dynamically generate video thumbnails and use them in a custom video player. Optimize your re-renders.

Every single interface has infinite opportunity for polish and delight. Reading that list is exciting – imagine a product that goes the extra mile, then goes an extra ten miles. It makes me want to drop everything spend my life crafting the details. I want to build extraordinary things.

The design review is the one meeting that is most dense in product decisions in the company; and since it is so early in a product’s lifecycle, it is when it is cheapest to make changes.

Frank Lloyd Wright said that “you can use an eraser on the drafting board, or a sledgehammer on the construction site.” The design review is the drafting board — don’t hesitate to make things take a bit longer at this phase. An extra 3 days here could save us 3 weeks down the line.

During these reviews, do:

- i. **Insist on reviewing whole journeys**, not just screens — we design experiences, not individual steps!

In the spirit of “staying close to the medium,” we try to put ourselves in conditions that are as close as possible to what the user will experience. This means that a good design review is spent *clicking on prototypes*, not *looking at designs*.

- ii. **Demand designs that use real data.** That includes copywriting: we unfortunately don’t have copywriters yet at Lindy — copywriting is the designer’s job, and we select for it when interviewing.
- iii. **Try prototypes yourself**, instead of letting designers walk you through the designs — they won’t be in the room with users!

And when trying these prototypes: do listen to your inner voice, notice the small bits of confusion (and magnify them 100x — if you’re confused, imagine how users will feel!) and leave comments in the Figma as you go along, before you forget.

- iv. **Ask for multiple explorations.** First, as a forcing function for the designers to have done these explorations in the first place — the odds are very slim that they stumbled upon the perfect solution first shot. Second, because tradeoffs appear more clearly once you compare options.
- v. **Sweat the details.** Do pay attention to every margin, every color, every opportunity to simplify, streamline, make more consistent. Do not be afraid of being nitpicky or stepping on toes — our job isn’t to be popular, but to build a “holy shit” product.
- vi. **Talk about the big things first.** While we absolutely do want to, again, sweat the details, it is best to leave the feedback on cosmetics for the end, and start with the bigger stuff. Did we get the flow right? Are we missing something big? Do notice when the team gets stuck [bikeshedding](#).

There is an implicit list of requirements from the designer here: designs presented at design reviews must have 1/ multiple explorations 2/ a proper Figma prototype set up 3/ real data, not placeholders.

If you find at the beginning of the meeting that any of these ingredients is missing, simply adjourn the meeting and have it at a time when we have everything we need to do make A+ decisions.

- 3. **In product reviews**, when we review work that is either in progress (so we detect and fix defects early) or about to get shipped (and requires product’s greenlight).

Product reviews are team meetings: we review your work in front of everyone else with something to show that week. There are a few reasons for this: first, it makes the meeting feel higher stakes — which it should! This is important — we’re about to put this in the

hands of our users!

Second, it lets everyone learn from the feedback that we give everyone else, and creates common knowledge about our quality bar.

Running the product review is one of your most important responsibilities — it is when the rubber meets the road! It doesn't matter how much time we spend on the previous steps, if we don't actually build what we designed.

What running it entails:

- i. *Before the meeting*, ensure that everyone who should present, will present. Every project DRI should have something to present.
- ii. *During the meeting*, again, insist on *trying the product yourself*, and *not* on watching the DRI demo it.

You might have noticed how e.g. video games seem so easy, when you're watching an expert play.

A software product is the same, and there is no greater expert than the person who built it. You will be a lot more engaged if you try the product yourself, and would be shocked at how many more issues you will catch.

Here again, you must be the keeper of product quality — the buck stops at you. If we miss something in this meeting, this is it — it's shipping to the user. Pay attention to the:

- a. Copy
- b. Margins
- c. Text sizes
- d. Border radii
- e. Colors
- f. Animations
- g. Zero states
- h. Error states
- i. Loading states

This list is not meant as a way to say “look closely at all the things” — I do mean that I literally want you to watch out for each of these specific, individual things.

Take copious notes in the product review’s running notes doc, with all the things you found in this meeting that should be fixed.

Then, refer back to these notes at the beginning of the next meeting, to make sure that all the things were addressed.

4. Finally, day to day — you will, hopefully, use Lindy quite a bit every day at work. As you do so, you will run into lots of small issues. Please do file tickets for them, using the Just Do It tag for the very small ones and the Fit & Finish Friday tag for things that should be fixed under 7 days.

Two utilities that you should install to make this easier are [Retroclip](#) and [CleanShot](#).

Retroclip constantly records your screen, so when you see an issue, you can press Shift + Cmd + 2, grab the video, and put it in a Linear ticket.

CleanShot is a more powerful screenshot tool letting you annotate screenshots, so it’s very clear what you find wrong in any given screenshot.

One failure mode we sometimes fall into here is that we go through all the steps above, *including* catching the small issues in the final product — and then never actually go around to fixing them!

That is because after a long time working on a project, the team is often looking forward to move onto something less. There are two ways you can help here.

First — help us catch it when we make this mistake. The one meeting where we do this is the roadmap review, where we decide on what we work on next. This is when we must hold the bar and point out that a given project still needs some love before we move on to the next thing.

Second — do what it takes for the team to actually prioritize these Fit & Finish Tickets. File the tickets in the first place; snooze them on Linear to check if they were done 2 days later; protect the team against distractions; and ensure that they present their work at the next product review, until there are no more notes.

“Build it Fast”

This is the “project manager” side of the job, which is already half-assumed by each project’s DRI (Directly Responsible Individual). Some PMs resent having to do this — we take a different view, and think no work is beneath us to make sure we ship this “holy shit” product.

You can help ensure we keep moving fast by:

1. **Making sure that the project is divided up into milestones**, no milestone being more than 2 weeks long.
2. **Making sure that we're on track to hit the next milestone on time.** To do that, bring up the exact date of this milestone every time we're talking of the project. "Do we still think we can make Jan 12th?" If the answer is no, make adjustments as soon as possible. We have a few options here:
 - a. Take some things off the plate of the engineers working on the feature — e.g. taking them off the on-call rotation. This is my favorite option — everyone can always benefit from more focus.
 - b. Decide that the project is going to take longer. My second favorite option, but when we do that, it's important that we at least *have a conversation about exactly what happened*.

The default state of any team is to slow down, and there is no limit to how inefficient a team can be (just ask the government). If we want to build a high-performance environment, it's important that we build the discipline to set ambitious goals, consistently hit them, and when we fail to do so, look back to see what we could have done better.

- c. Cut the scope — we try to avoid doing this since, again, we believe in shipping great products, and would rather things take a bit longer.

When we do cut scope, it's important that we first check the job to be done that we defined during the product jam: it seems obvious, but teams often compromise so much while executing that they cut too deep into the very "why" of the product, and end up shipping something that's missing the mark.

3. **Helping prioritize bugs.** Read [What We Should Fix When](#) on how we think about bug triage.
4. **Driving urgency.** Read "[Speed as a Habit](#)." The TL;DR is that speed is a decision you make every day, multiple times a day — a lot of small actions that compound into your moving 10x faster than your competitor.
 - a. When you see a discussion go on for too long over Slack / Gdocs / Figma / Linear comments, bring its participants together in a room instead. Don't schedule a meeting! Literally just get up from your chair, poke people on the shoulder, and have a conversation.
 - b. When you see someone leave a comment waiting for someone else's answer on any channel but Slack (again — Gdocs, Figma, Linear...), bring it over to Slack —

people miss email notifications all the time, and even when they see them, they are so much more responsive over Slack. You just sped up the project by a couple of hours!

- c. When you notice that any part of a given project could be confusing, have an IRL conversation with the team about it.

Nothing comes close to synchronous conversations in front of a whiteboard to drive clarity and alignment. Then, document the conversation and circulate it around — that part can happen over Slack or Linear.

- d. When you see a large meeting get put on the books 4 days from now, ask whether it could be a smaller huddle in the office, happen today, and have fewer people.
- e. Decline meetings that don't seem like a good use of time, are too large, or have unclear agendas.

5. Bringing order to the chaos.

- a. Document thoroughly every decision we make and keep them all in a single source of truth for each project.

It sounds easy, but we make *a lot* of product decisions all day, be it in meetings, ad hoc syncs in the office, random Slack threads, etc... You gotta catch them all (🎵), write them down, circulate them, and make sure we follow through on them.

- b. Make sure that people have access to all the resources they need and know exactly where to find them.

Peter Drucker identified “malfunctioning information systems” as one of the greatest time-wasters in organizations. People either waste time looking for information; or, worse, give up on even finding the information, and as a result do things they must later unwind.

There is an easy, concrete solution: make sure that *every resource* pertaining to a project is in the header or the bookmarks of that project's Slack Channel:

project-testing



Messages Running Notes Files **Bookmarks** +

Add Bookmark



Jam

www.figma.com



Designs (old)

www.figma.com



Designs (latest)

www.figma.com



Project

linear.app



Running Notes

docs.google.com



PostHog Dashboard

us.posthog.com

If you see anyone look for a resource that is not in there, do add it after you found it.

- c. Set *lots* of reminders on the things you care about. The downside of asynchronous collaboration is that balls fall through the cracks all the time — it's on you to ensure that when you send a ball, *someone catches it*.

It's hard to over-emphasize how important that is — Andy Grove said that the only difference between delegation and abandonment was in monitoring. If someone drops a ball, it's their fault; but if you don't *notice* that they dropped it, it's your fault.

Concretely, that means that every meaningful loop you open — every message on Slack, every comment on Gdoc, Linear or Figma — must come with a reminder in your personal todo. If you do this right, you should end up creating about 10 such reminders every day.

Your First Few Weeks

Week 1

- Get setup
- Do everything on the onboarding checklist
- Create a couple of Lindies of your own
 - Keep a friction log throughout, and share it with the team at the end of the week
- Get the list of top 20 customers from X
- Set up a Lindy that reaches out to them, sends follow up emails if no reply, and schedules time with them
- Join the customer support rotation and answer 20+ tickets
- [Join the Lindy community](#) and answer 20+ questions
- Have 1:1s with: ...
- Set up shadowing sessions for X's implementation calls

Week 2

- Have meetings with first customers
 - Invite Flo to all of them
 - Take plenty of notes & share them with the team
- Shadow more calls with X
- Lead the "X" project
 - It's a very simple one
 - Must mostly work with X to get designs and with Y (who'll be DRI) to make sure he gets what needs to be done — then ensure we get it done that week
- Write and publish your first changelog
- Help write the documentation for the phone calls project

Week 3

- Lead your first product review (if you feel ready)
- Define + implement plan to measure user satisfaction on an ongoing basis
 - What question do we ask? Who do we ask it to? At what point of their user journey? Through what channel?